



DATA ANALYSIS & AI FUNDAMENTALS

Trainer: Lumumba W. Victor



Data Visualization and Insights Extraction

*A Comprehensive Guide to Univariate, Bivariate, and
Multivariate Analysis*

Course: Data Analytics and Fundamentals of AI

Module: Week 3 - Day 2

June 30, 2026

Prepared using Python, Pandas, Matplotlib, and Seaborn

Topics Covered: EDA, Statistical Plotting, Correlation, Advanced Dashboards

Contents

1	Introduction to Exploratory Data Analysis (EDA)	2
1.1	The Role of Visualization	2
1.2	The Dataset: General Social Survey (GSS)	2
2	Environment Setup and Data Ingestion	3
2.1	Importing Libraries	3
2.2	Configuring Visualization Styles	3
2.3	Data Importation and Initial Inspection	3
3	Univariate Analysis: Continuous Variables	5
3.1	Histograms and Kernel Density Estimation (KDE)	5
3.1.1	Understanding Skewness	5
3.2	Box Plots: The Five-Number Summary	6
3.3	Violin Plots	6
4	Univariate Analysis: Categorical Variables	8
4.1	Frequency Tables	8
4.2	Bar Charts	8
4.3	Horizontal Bar Charts	8
4.4	Pie Charts	9
5	Bivariate Analysis: Continuous vs. Continuous	10
5.1	Scatter Plots and Regression Lines	10
5.2	Joint Plots	10
5.3	Pearson Correlation Coefficient	11
6	Bivariate Analysis: Categorical vs. Continuous	12
6.1	Bar Plots with Estimators and Error Bars	12
6.1.1	Standard Deviation vs. Confidence Intervals	12
6.2	Grouped Bar Charts (Using hue)	12
7	Multivariate Analysis	14
7.1	Correlation Heatmaps	14
7.2	Pairplots (Scatterplot Matrix)	14
8	Advanced Plotting and Custom Dashboards	16
8.1	Creating a 2x2 Dashboard	16
8.2	Customization Best Practices	17
9	Conclusion and Best Practices	18
9.1	Choosing the Right Plot	18
9.2	Avoiding Misleading Visualizations	18
9.3	Final Thoughts	18
10	Appendix: Quick Reference Guide	20
10.1	Essential Seaborn Functions	20
10.2	Essential Matplotlib Customizations	20

1 Introduction to Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is a crucial step in the data science pipeline. Before building complex machine learning models, data scientists must understand the underlying structure, anomalies, and patterns within the data. Data visualization is the primary tool used in EDA to transform raw numerical and categorical data into intuitive graphical representations.

1.1 The Role of Visualization

Visualization serves several critical purposes:

- **Pattern Recognition:** Identifying trends, seasonality, and clusters that are invisible in tabular data.
- **Outlier Detection:** Spotting extreme values that could skew model predictions.
- **Assumption Checking:** Verifying statistical assumptions (e.g., normality, homoscedasticity) required for various algorithms.
- **Communication:** Conveying complex data insights to non-technical stakeholders effectively.

1.2 The Dataset: General Social Survey (GSS)

In these notes, we will analyze a subset of the General Social Survey (GSS). The dataset contains demographic and socioeconomic information about individuals.

Dataset Variables

The dataset `GSSsubset.csv` contains the following variables:

- **id:** Unique identifier for the respondent.
- **sex:** Gender of the respondent (Categorical: MALE, FEMALE).
- **degree:** Highest educational qualification (Categorical: LT HIGH SCHOOL, HIGH SCHOOL, JUNIOR COLLEGE, BACHELOR, GRADUATE).
- **income:** Annual income in USD (Continuous).
- **marital:** Marital status (Categorical: MARRIED, DIVORCED, etc.).
- **age:** Age in years (Continuous/Discrete).
- **height:** Height in inches (Continuous).
- **weight:** Weight in pounds (Continuous).
- **hrswrk:** Hours worked per week (Continuous/Discrete).

2 Environment Setup and Data Ingestion

Before visualizing data, we must set up our Python environment. The primary libraries used in this module are `pandas` for data manipulation, `numpy` for numerical operations, `matplotlib` for foundational plotting, and `seaborn` for high-level statistical graphics.

2.1 Importing Libraries

We begin by importing the necessary libraries and suppressing non-critical warnings to keep the output clean.

```
1 # Import core data manipulation libraries
2 import pandas as pd          # For data importation and dataframe manipulation
3 import numpy as np          # For numeric manipulation and mathematical functions
4
5 # Import visualization libraries
6 import seaborn as sns       # High-level statistical data visualization
7 import matplotlib.pyplot as plt # Foundational plotting library
8
9 # Import warnings to ignore minor computational warnings
10 import warnings
11 warnings.filterwarnings('ignore')
```

Listing 1: Importing Libraries and Setup

2.2 Configuring Visualization Styles

Consistency in visual presentation is key to professional data analysis. We use Seaborn's built-in themes and Matplotlib's `rcParams` to set global defaults for figure sizes and font sizes.

```
1 # Set the aesthetic style of the plots
2 sns.set_style("whitegrid") # Adds a clean grid background
3
4 # Set default figure size and font size globally
5 plt.rcParams["figure.figsize"] = [10, 6] # Width: 10, Height: 6
6 plt.rcParams["font.size"] = 10          # Base font size for text elements
```

Listing 2: Setting Global Visualization Styles

2.3 Data Importation and Initial Inspection

Loading the data is the first practical step. We use the `pandas.read_csv()` function. Immediately after loading, it is mandatory to inspect the data to ensure it was read correctly.

```
1 # Load the dataset
2 df = pd.read_csv("GSSsubset.csv")
3
4 # Display the first 5 rows to check data structure
5 display(df.head())
```

```
6
7 # Check dimensions (rows, columns)
8 print(f>Data Dimensions: {df.shape}<
9
10 # Check data types and missing values
11 df.info()
```

Listing 3: Loading and Inspecting Data

The Importance of `df.info()`

The `df.info()` method is arguably the most important initial check. It reveals:

1. **Memory Usage:** Helps determine if the dataset will fit in RAM.
2. **Column Data Types:** Ensures numerical columns are not mistakenly loaded as strings (objects), which would prevent mathematical operations.
3. **Non-Null Counts:** Instantly highlights missing data (NaN values) that require imputation or removal.

3 Univariate Analysis: Continuous Variables

Univariate analysis involves examining a single variable at a time. For continuous variables, the primary goals are to understand the **distribution**, identify the **central tendency** (mean, median), measure **dispersion** (standard deviation, range), and detect **skewness** and **outliers**.

3.1 Histograms and Kernel Density Estimation (KDE)

A histogram divides the range of continuous data into discrete intervals (bins) and counts the number of observations falling into each bin.

```
1 plt.figure(figsize=(14, 8))
2 # Create histogram with KDE overlay
3 sns.histplot(df['income'], bins=20, kde=True, color="skyblue", edgecolor="black"
4             )
5 plt.title("Histogram Showing the Distribution of Income", fontsize=14,
6           fontweight="bold")
7 plt.xlabel("Income ($)")
8 plt.ylabel("Frequency")
9 # Add vertical lines for Mean and Median
10 plt.axvline(df['income'].median(), color='red', linestyle='--',
11             label=f'Median: ${df["income"].median():.0f}')
12 plt.axvline(df['income'].mean(), color='black', linestyle='--',
13             label=f'Mean: ${df["income"].mean():.0f}')
14
15 plt.legend()
16 plt.show()
```

Listing 4: Histogram of Income with KDE

3.1.1 Understanding Skewness

Comparing the mean and median is a quick way to identify the skewness (asymmetry) of a distribution. The mean is highly sensitive to extreme outliers, while the median represents the robust middle point.

Types of Skewness

- **Zero Skew (Symmetrical):** Mean $\approx$ Median. The distribution resembles a classic bell curve.
- **Right (Positive) Skew:** Mean $>$ Median. The tail extends toward higher positive values. Extreme high values pull the mean up. *Income data is typically right-skewed.*
- **Left (Negative) Skew:** Mean $<$ Median. The tail extends toward lower negative values. Extreme low values pull the mean down.

3.2 Box Plots: The Five-Number Summary

While histograms show the shape of the distribution, box plots provide a concise visual summary based on percentiles.

```

1 plt.figure(figsize=(10, 6))
2 sns.boxplot(y=df['income'], color="lightblue")
3
4 plt.title("The Five Summary Statistics for Income", fontsize=14, fontweight='
    bold')
5 plt.ylabel("Income ($)")
6
7 # Overlay Mean and Median lines
8 plt.axhline(df['income'].mean(), color="red", linestyle='--', alpha=0.9, label='
    Mean')
9 plt.axhline(df['income'].median(), color="blue", linestyle='--', alpha=0.9,
    label='Median')
10
11 plt.legend()
12 plt.tight_layout()
13 plt.show()

```

Listing 5: Box Plot of Income

Anatomy of a Box Plot

- **The Box:** Represents the Interquartile Range (IQR), containing the middle 50% of the data (from the 25th percentile Q_1 to the 75th percentile Q_3).
- **The Line inside the Box:** The Median (Q_2).
- **The Whiskers:** Extend to $1.5 \times \text{IQR}$ from the hinges of the box. They represent the range of the "bulk" of the data.
- **The Outliers:** Points plotted beyond the whiskers. These are extreme values that may require investigation.

3.3 Violin Plots

A violin plot is a combination of a box plot and a Kernel Density Estimate (KDE). It shows the full distribution of the data, including its density at different values, rather than just summary statistics.

```

1 fig, axes = plt.subplots(1, 2, figsize=(14, 6))
2
3 # Left: Histogram
4 sns.histplot(df['age'], bins=20, kde=True, ax=axes[0], color='steelblue',
    edgecolor='black')
5 axes[0].set_title("Age Distribution using Histogram", fontweight='bold')
6
7 # Right: Violin Plot
8 sns.violinplot(y=df['age'], ax=axes[1], color='lightgreen')
9 axes[1].set_title("Age Distribution using Violin Plot", fontweight='bold')
10
11 plt.tight_layout()

```

```
12 plt.show()
```

Listing 6: Violin Plot of Age

The width of the "violin" indicates the density of data at that specific value; wider sections mean more observations are clustered around that value.

4 Univariate Analysis: Categorical Variables

Categorical variables represent characteristics or groups (e.g., Gender, Degree, Marital Status). The primary goal of analyzing these variables is to understand the **frequency** or **proportion** of observations within each category.

4.1 Frequency Tables

Before plotting, it is essential to calculate the raw counts using the `value_counts()` method in Pandas.

```
1 # Calculate and sort frequencies
2 frequency_table = df['degree'].value_counts().sort_values()
3 print(frequency_table)
```

Listing 7: Frequency Table for Degree

4.2 Bar Charts

Bar charts are the standard visualization for categorical data. The height of each bar corresponds to the count or proportion of the category.

```
1 plt.figure(figsize=(10,6))
2 ax = sns.countplot(data=df, x="degree", palette='Set1', edgecolor='black',
3     linewidth=1.5)
4 plt.title("Distribution of Degree Qualification", fontsize=14, fontweight='bold'
5     )
6 plt.xlabel("Degree")
7 plt.ylabel("Frequency")
8 # Adding percentage labels on top of bars
9 total = len(df)
10 for p in ax.patches:
11     percentage = f'{100 * p.get_height() / total:.1f}%'
12     x = p.get_x() + p.get_width() / 2
13     y = p.get_height() + 10
14     ax.annotate(percentage, (x, y), ha='center', fontweight='bold')
15
16 plt.tight_layout()
17 plt.show()
```

Listing 8: Vertical Bar Chart with Percentage Annotations

4.3 Horizontal Bar Charts

When category names are long (e.g., "LT HIGH SCHOOL"), horizontal bar charts prevent label overlapping and improve readability.

```
1 plt.figure(figsize=(10,6))
2 degree_counts = df['degree'].value_counts()
```

```
3
4 # Note: x and y are swapped compared to vertical bar charts
5 ax = sns.barplot(x=degree_counts.values, y=degree_counts.index, palette='Set2',
6                 edgecolor='red')
7
8 plt.title("Frequency of Educational Qualification", fontweight='bold')
9 plt.xlabel("Frequency")
10 plt.ylabel("Degree Qualification")
11
12 # Add text labels at the end of the bars
13 for i, v in enumerate(degree_counts.values):
14     ax.text(v + 10, i, str(v), va='center', fontweight='bold')
15
16 plt.tight_layout()
17 plt.show()
```

Listing 9: Horizontal Bar Chart

4.4 Pie Charts

Pie charts show the proportion of categories as slices of a circle. While popular in business, they are often discouraged in strict scientific contexts because humans are better at judging length (bar charts) than angles (pie charts).

```
1 plt.figure(figsize=(10, 8))
2 degree_counts = df['degree'].value_counts()
3 colors = plt.cm.Set3(np.linspace(0, 1, len(degree_counts)))
4
5 plt.pie(
6     degree_counts,
7     labels=degree_counts.index,
8     autopct='%1.1f%%',      # Format string for percentages
9     colors=colors,
10    startangle=90,           # Rotate the start of the pie
11    explode=[0.05] * len(degree_counts) # Slightly separate all slices
12 )
13
14 plt.title("Distribution of Degree Qualification", fontsize=14, fontweight='bold')
15
16 plt.axis('equal') # Ensures the pie is drawn as a circle
17 plt.show()
```

Listing 10: Pie Chart of Degree Qualification

5 Bivariate Analysis: Continuous vs. Continuous

Bivariate analysis involves examining two variables simultaneously to determine if there is a relationship or association between them. When both variables are continuous, we look for correlations and trends.

5.1 Scatter Plots and Regression Lines

A scatter plot places one variable on the X-axis and the other on the Y-axis. Each point represents an observation. Adding a regression line helps visualize the overall trend.

```
1 plt.figure(figsize=(10, 6))
2 # sns.regplot automatically calculates and plots the linear regression line
3 sns.regplot(
4     data=df,
5     x='height',
6     y='weight',
7     scatter_kws={'alpha': 0.6}, # Transparency for overlapping points
8     line_kws={'color': 'red', 'linewidth': 2} # Style the regression line
9 )
10
11 plt.title("Relationship Between Weight and Height", fontsize=14, fontweight="
    bold")
12 plt.xlabel("Height (inches)")
13 plt.ylabel("Weight (pounds)")
14 plt.tight_layout()
15 plt.show()
```

Listing 11: Scatter Plot with Regression Line

5.2 Joint Plots

A joint plot is an advanced visualization that combines a central scatter plot with marginal histograms (or KDE plots) on the top and right edges. This allows you to see the bivariate relationship and the univariate distributions simultaneously.

```
1 joint_plot = sns.jointplot(
2     data=df,
3     x="height",
4     y="weight",
5     kind='scatter',
6     alpha=0.7,
7     color='steelblue',
8     marginal_kws=dict(bins=30, fill=True)
9 )
10
11 joint_plot.fig.suptitle('Joint Distribution of Weight and Height', fontsize=14,
    fontweight='bold', y=1.03)
12 plt.tight_layout()
13 plt.show()
```

Listing 12: Joint Plot of Height and Weight

5.3 Pearson Correlation Coefficient

To quantify the linear relationship between two continuous variables, we calculate the Pearson correlation coefficient (r).

Mathematical Definition of Pearson's r

The Pearson correlation coefficient is defined as the covariance of the two variables divided by the product of their standard deviations:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (1)$$

Interpretation:

- $r = 1$: Perfect positive linear relationship.
- $r = 0$: No linear relationship.
- $r = -1$: Perfect negative linear relationship.
- $|r| > 0.7$: Generally considered a strong correlation.

```
1 corr_val = df['height'].corr(df['weight'])  
2 print(f"Pearson Correlation between Height and Weight: {corr_val:.3f}")
```

Listing 13: Calculating Correlation

In our dataset, the correlation is approximately **0.482**, indicating a moderate positive linear relationship.

6 Bivariate Analysis: Categorical vs. Continuous

When analyzing one categorical variable and one continuous variable, the goal is to compare the distribution or central tendency of the continuous variable across the different categories.

6.1 Bar Plots with Estimators and Error Bars

By default, Seaborn's `barplot` calculates the **mean** of the continuous variable for each category. It is crucial to include error bars to understand the variance or confidence within those categories.

```
1 plt.figure(figsize=(10, 6))
2 sns.barplot(
3     data=df,
4     x='degree',
5     y='income',
6     estimator='mean',    # Statistic to plot
7     errorbar='sd',      # Error bar represents Standard Deviation
8     palette='Set1'
9 )
10
11 plt.title("Average Income by Degree Qualification", fontsize=14, fontweight='
12     bold')
13 plt.xlabel("Degree")
14 plt.ylabel("Mean Income ($)")
15 plt.xticks(rotation=0)
16 plt.tight_layout()
17 plt.show()
```

Listing 14: Mean Income by Degree with Standard Deviation

6.1.1 Standard Deviation vs. Confidence Intervals

- **Standard Deviation** (`errorbar='sd'`): Shows the spread of the *data*. Tall error bars mean the individual incomes vary wildly within that degree group.
- **Confidence Interval** (`errorbar=('ci', 95)`): Shows the precision of the *mean estimate*. It tells us where the true population mean is likely to fall.

6.2 Grouped Bar Charts (Using hue)

To compare a continuous variable across two categorical variables simultaneously, we use the `hue` parameter. This creates side-by-side bars for each subgroup.

```
1 plt.figure(figsize=(12, 7))
2 ax = sns.barplot(
3     data=df,
4     x="degree",
5     y="income",
6     hue="sex",          # Split by gender
7     estimator="mean",
```

```
8     errorbar("ci", 95),    # 95% Confidence Interval
9     palette="viridis"
10 )
11
12 ax.set_title("Mean Income by Educational Degree and Sex", fontsize=16,
13             fontweight="bold")
14 ax.set_xlabel("Educational Degree", fontsize=12)
15 ax.set_ylabel("Mean Income ($)", fontsize=12)
16
17 plt.legend(title="Gender", title_fontsize=12, fontsize=11)
18 sns.despine() # Remove top and right spines for cleaner look
19 plt.tight_layout()
20 plt.show()
```

Listing 15: Grouped Bar Chart: Income by Degree and Gender

7 Multivariate Analysis

Multivariate analysis involves observing and analyzing more than two variables simultaneously to understand the relationships and patterns within a complex dataset.

7.1 Correlation Heatmaps

When dealing with multiple continuous variables, calculating pairwise correlations manually is tedious. A correlation heatmap provides a color-coded matrix of correlation coefficients.

```
1 # Select only continuous variables for correlation
2 continuous_vars = ['age', 'income', 'hrswrk', 'height', 'weight']
3
4 # Compute the Pearson correlation matrix
5 corr_matrix = df[continuous_vars].corr(method='pearson')
6
7 plt.figure(figsize=(8, 6))
8 sns.heatmap(
9     corr_matrix,
10    annot=True,          # Display the correlation numbers inside the cells
11    cmap='coolwarm',     # Color map: Blue (negative) to Red (positive)
12    fmt='.2f',           # Format to 2 decimal places
13    linewidths=0.5,     # White lines between cells
14    square=True,        # Make cells square
15    cbar_kws={'label': 'Pearson Correlation'}
16 )
17
18 plt.title('Correlation Heatmap of Continuous Variables', fontsize=14, fontweight
19          = 'bold')
20 plt.xticks(rotation=45)
21 plt.tight_layout()
22 plt.show()
```

Listing 16: Correlation Heatmap

Reading a Heatmap

- **Dark Red:** Strong positive correlation (as X increases, Y increases).
- **Dark Blue:** Strong negative correlation (as X increases, Y decreases).
- **White/Light Color:** No linear correlation.
- **The Diagonal:** Always 1.00, because a variable is perfectly correlated with itself.

7.2 Pairplots (Scatterplot Matrix)

A pairplot is an exhaustive visualization tool. It creates a grid of scatter plots for every possible pair of continuous variables. The diagonal of the grid displays the univariate distribution (histograms or KDE) of each individual variable.

```
1 continuous_vars = ['age', 'income', 'hrswrk', 'height', 'weight']
2
```

```
3 # Create the pairplot
4 pair = sns.pairplot(
5     df[continuous_vars],
6     diag_kind='hist', # Histograms on the diagonal
7     corner=True,      # Only plot the lower triangle to save space
8     plot_kws={
9         'alpha': 0.6,
10        's': 40,        # Size of scatter points
11        'edgecolor': 'black'
12    },
13    diag_kws={
14        'bins': 20,
15        'edgecolor': 'black'
16    }
17 )
18
19 pair.fig.suptitle('Pairwise Relationships Among Continuous Variables',
20                  fontsize=16, fontweight='bold', y=1.02)
21 plt.show()
```

Listing 17: Pairplot of Continuous Variables

Pairplots are incredibly powerful for quickly spotting non-linear relationships, clusters, and outliers across an entire dataset.

8 Advanced Plotting and Custom Dashboards

In real-world scenarios, analysts often need to combine multiple plots into a single comprehensive dashboard. Matplotlib's `subplots` functionality allows for the creation of complex grid layouts.

8.1 Creating a 2x2 Dashboard

We will construct a 2x2 grid containing four different types of plots to summarize the dataset effectively.

```

1 # Initialize the 2x2 grid
2 fig, axes = plt.subplots(2, 2, figsize=(15, 12))
3 sns.set_style("whitegrid")
4
5 # Data preparation
6 degree_counts = df['degree'].value_counts()
7 colors = plt.cm.Set3(np.linspace(0, 1, len(degree_counts)))
8
9 # -----
10 # 1. Top-Left: Pie Chart
11 # -----
12 axes[0, 0].pie(
13     degree_counts,
14     labels=degree_counts.index,
15     autopct='%1.1f%%',
16     startangle=90,
17     colors=colors,
18     explode=[0.03] * len(degree_counts)
19 )
20 axes[0, 0].set_title("Distribution of Degree Qualification", fontweight='bold')
21
22 # -----
23 # 2. Top-Right: Horizontal Bar Chart
24 # -----
25 sns.barplot(
26     x=degree_counts.values,
27     y=degree_counts.index,
28     palette='Set2',
29     edgecolor='black',
30     ax=axes[0, 1]
31 )
32 axes[0, 1].set_title("Frequency of Degree Qualification", fontweight='bold')
33 axes[0, 1].set_xlabel("Frequency")
34 axes[0, 1].set_ylabel("Degree")
35
36 # Add text labels
37 for i, v in enumerate(degree_counts.values):
38     axes[0, 1].text(v + 2, i, str(v), va='center', fontsize=10)
39
40 # -----

```

```

41 # 3. Bottom-Left: Histogram of Income
42 # -----
43 sns.histplot(
44     data=df,
45     x='income',
46     bins=20,
47     kde=True,
48     color='steelblue',
49     edgecolor='black',
50     ax=axes[1, 0]
51 )
52 axes[1, 0].set_title("Distribution of Income", fontweight='bold')
53 axes[1, 0].set_xlabel("Income ($)")
54 axes[1, 0].set_ylabel("Frequency")
55
56 # -----
57 # 4. Bottom-Right: Boxplot of Income by Degree
58 # -----
59 sns.boxplot(
60     data=df,
61     x='degree',
62     y='income',
63     palette='Pastel1',
64     ax=axes[1, 1]
65 )
66 axes[1, 1].set_title("Income by Degree Qualification", fontweight='bold')
67 axes[1, 1].set_xlabel("Degree")
68 axes[1, 1].set_ylabel("Income ($)")
69 axes[1, 1].tick_params(axis='x', rotation=45)
70
71 # Adjust layout to prevent overlapping
72 plt.tight_layout()
73 plt.show()

```

Listing 18: Building a 2x2 Subplot Dashboard

8.2 Customization Best Practices

When building complex dashboards, adhere to the following principles:

1. **Consistent Color Palettes:** Use the same color for the same category across different plots (e.g., "BACHELOR" should be the same color in the pie chart and the boxplot).
2. **Descriptive Titles:** Every subplot must have a clear, descriptive title.
3. **Rotation of Labels:** If X-axis labels overlap (like degree names), rotate them using `tick_params(axis='x', rotation=45)`.
3. **Whitespace Management:** Always use `plt.tight_layout()` at the end to automatically adjust subplot parameters so labels don't overlap.

9 Conclusion and Best Practices

Data visualization is both an art and a science. Throughout this module, we have transitioned from basic univariate histograms to advanced multivariate pairplots and custom dashboards.

9.1 Choosing the Right Plot

The most critical skill in EDA is selecting the appropriate visualization for the data type and the analytical question:

Data Types	Goal	Recommended Plot
Continuous	Distribution	Histogram, KDE, Violin Plot
Continuous	Outliers	Box Plot
Categorical	Proportions	Bar Chart, Pie Chart
Continuous vs Continuous	Relationship	Scatter Plot, Joint Plot, Heatmap
Categorical vs Continuous	Comparison	Grouped Bar Plot, Box Plot
Multiple Continuous	Pairwise Relations	Pairplot, Correlation Matrix

9.2 Avoiding Misleading Visualizations

- **Truncated Y-Axis:** Starting the Y-axis at a value other than zero in bar charts can exaggerate minor differences.
- **Dual Y-Axes:** Using two different Y-axes on a single line chart can imply a causal relationship where none exists.
- **3D Plots:** Avoid 3D pie charts or bar charts; they distort proportions and make data difficult to read accurately.
- **Inconsistent Scales:** When comparing multiple bar charts, ensure they share the same scale.

9.3 Final Thoughts

The Python ecosystem, particularly the combination of **Pandas**, **Matplotlib**, and **Seaborn**, provides unparalleled flexibility for data exploration. By mastering these tools, you can uncover hidden insights, communicate findings effectively, and build a solid foundation for predictive modeling.

Summary of Key Libraries

- **Pandas:** Data manipulation, aggregation, and handling missing values.
- **Matplotlib:** The foundational library for creating customizable, static visualizations.
- **Seaborn:** Built on top of Matplotlib, provides beautiful default styles and complex statistical plots with minimal code.
- **NumPy:** Provides the underlying numerical arrays and mathematical functions used by Pandas and Seaborn.

10 Appendix: Quick Reference Guide

10.1 Essential Seaborn Functions

- `sns.histplot(data, x, bins, kde)`: Plot univariate distribution.
- `sns.boxplot(data, x, y)`: Plot five-number summary.
- `sns.violinplot(data, x, y)`: Plot distribution density.
- `sns.countplot(data, x)`: Plot categorical counts.
- `sns.barplot(data, x, y, estimator, errorbar)`: Plot point estimates and confidence intervals.
- `sns.scatterplot(data, x, y, hue)`: Plot bivariate relationships.
- `sns.regplot(data, x, y)`: Scatter plot with linear regression.
- `sns.jointplot(data, x, y)`: Bivariate scatter with marginal univariate distributions.
- `sns.pairplot(data)`: Matrix of bivariate scatter plots.
- `sns.heatmap(data, annot, cmap)`: Plot correlation matrices.

10.2 Essential Matplotlib Customizations

- `plt.title("Text", fontsize, fontweight)`: Set plot title.
- `plt.xlabel("Text")` / `plt.ylabel("Text")`: Set axis labels.
- `plt.axvline(x)` / `plt.axhline(y)`: Add vertical/horizontal reference lines.
- `plt.legend()`: Display the legend.
- `plt.tight_layout()`: Automatically adjust subplot spacing.
- `plt.show()`: Render the plot.